Studium
Enciclopedia de Programación e Ingeniería del Software

Volumen IV: Monografías de Lenguajes de Sistemas — C, C++, Zig, Ensamblador, Ada y Fortran

Parte V (Bloques A–H): Memoria manual, RAII, metaprogramación con Concepts, ABI de bajo nivel y cómputo científico

Borrador
Edición 0.1.0 — No publicado
20 de octubre de 2026

Prefacio

Este libro presenta el tema con el lenguaje del curso. Cada capítulo abre con una entrada breve, sigue con la explicación y cierra con un problema resuelto y una autoficha.

Cómo usar este libro

Lee el capítulo seguido. El consejo, la definición, el problema resuelto y la autoficha van en recuadros. La explicación es el cuerpo del texto.
Un capítulo lleva una entrada en cursiva, la explicación en el cuerpo, como mucho un consejo, definiciones solo al introducir un término, un problema resuelto y una autoficha.

Índice

Prefacio II

Cómo usar este libro III

I Monografía Autónoma de C (ISO/IEC 9899:2018 / C23) 1

1. Monografía de C (I – Bloques A, B y C): Historia, modelo de traducción ISO C17/C23, toolchain y nivel principiante 2
1.1. Bloques A y B: Historia de C, modelo de máquina abstracta, GCC/Clang, CMake y flags de diagnóstico . . . 2
1.2. Bloque C: Tipos fundamentales, <stdint.h>, promociones aritméticas, operadores y control de flujo . . . 3

2. Monografía de C (II – Bloques D y E): Punteros, aritmética de direcciones, memoria dinámica, structs y strict aliasing 5
2.1. Bloque D: Punteros, arrays, cadenas terminadas en NUL, structs, unions y memoria dinámica 5
2.2. Bloque E: Punteros a función, calificador restrict, strict aliasing y asignadores de región (Arena) . . . 5

3. Monografía de C (III – Bloques F, G y H): Concurrencia C11/POSIX, Sanitizers, Ecosistema y Proyectos 8
3.1. Bloque F: Concurrencia C11 (<stdatomic.h>), pthreads, llamadas POSIX y programación defensiva . . . 8
3.2. Bloques G y H: Ecosistema (CMake, Meson, pkg-config, Sanitizers) y los 4 proyectos en C 9

II Monografía Autónoma de C++ (ISO/IEC 14882:2020 / C++23) 11

4. Monografía de C++ (I – Bloques A–D): De C++98 a C++20/23, RAII, Semántica de Movimiento y Punteros Inteligentes 12
4.1. Bloques A–C: Evolución C++11/14/17/20/23, toolchain, referencias, sobrecarga, constexpr y RAII . . . 12
4.2. Bloque D: Categorías de valores (lvalue/rvalue), Regla de Cero/Cinco, std::unique_ptr y std::span . . . 12

5. Monografía de C++ (II – Bloques E–H): Plantillas, Concepts C++20, std::ranges, Concurrencia, Corrutinas y Proyectos 15
5.1. Bloques E y F: Plantillas, C++20 Concepts, std::ranges, std::jthread y corrutinas . . . 15
5.2. Bloques G y H: Ecosistema CMake/vcpkg/Conan/Clang-Tidy y los 4 proyectos (incluyendo cpp-task-scheduler) . . . 16

III Monografías Especializadas de Sistemas: Zig, Ensamblador, Ada y Fortran 18

6. Monografía de Zig (Bloques A–H): Asignadores explícitos, metaprogramación comptime, uniones de error y toolchain C 19
6.1. Bloques A–D: Filosofía de Zig, toolchain integrada, tipos, rebanadas, uniones de error y defer/errdefer . . . 19
6.2. Bloques E–H: Asignadores explícitos (Arena, FixedBuffer, GPA), comptime, FFI con C y proyectos . . . 20

7. Monografía de Ensamblador ARM64 (AArch64) y x86-64 (Bloques A−H): Registros, ABI, Pila y SIMD **22**
7.1. Bloques A−D: Arquitectura de registros x86-64 y ARM64 (AArch64), modos de direccionamiento y saltos . . . 22
7.2. Bloques E−H: Convenciones ABI (System V y AAPCS64), alineamiento de pila, SIMD (NEON/AVX) e inspección con objdump . . . 23

8. Monografías de Ada/SPARK (Sistemas Críticos) y Fortran Moderno (Cómputo Científico HPC) (Bloques A−H) **25**
8.1. Monografía de Ada 2022 y SPARK (Bloques A−H): Tipado fuerte de dominio, contratos y sistemas críticos . . . 25
8.2. Monografía de Fortran 2018/2023 (Bloques A−H): Arrays multidimensionales, vectorización y Coarrays . . . 26

A. Notación **28**

B. Hoja de fórmulas **29**

C. Soluciones **30**
C.1. Monografía de C (I − Bloques A, B y C): Historia, modelo de traducción ISO C17/C23, toolchain y nivel principiante . . . 30
C.2. Monografía de C (II − Bloques D y E): Punteros, aritmética de direcciones, memoria dinámica, structs y strict aliasing . . . 30
C.3. Monografía de C (III − Bloques F, G y H): Concurrencia C11/POSIX, Sanitizers, Ecosistema y Proyectos . . . 31
C.4. Monografía de C++ (I − Bloques A−D): De C++98 a C++20/23, RAII, Semántica de Movimiento y Punteros Inteligentes . . . 32
C.5. Monografía de C++ (II − Bloques E−H): Plantillas, Concepts C++20, std::ranges, Concurrencia, Corrutinas y Proyectos . . . 32
C.6. Monografía de Zig (Bloques A−H): Asignadores explícitos, metaprogramación comptime, uniones de error y toolchain C . . . 33
C.7. Monografía de Ensamblador ARM64 (AArch64) y x86-64 (Bloques A−H): Registros, ABI, Pila y SIMD . . . 34
C.8. Monografías de Ada/SPARK (Sistemas Críticos) y Fortran Moderno (Cómputo Científico HPC) (Bloques A−H) . . . 34

D. Auditoría de fuentes **36**

E. Plan de estudio **38**

Parte I

Monografía Autónoma de C (ISO/IEC 9899:2018 / C23)

Capítulo 1

Monografía de C (I – Bloques A, B y C): Historia, modelo de traducción ISO C17/C23, toolchain y nivel principiante

Este capítulo inicia la monografía autónoma del lenguaje C (Bloques A, B y C). C es la lingua franca de los sistemas operativos (Linux, Windows NT, XNU/macOS), los motores de bases de datos (PostgreSQL, SQLite, Redis), los intérpretes de lenguajes (CPython, Lua) y los sistemas empotrados. El lector aprenderá a programar en C moderno (C17/C23) desde cero con disciplina de tipos y herramientas de verificación rigurosas.

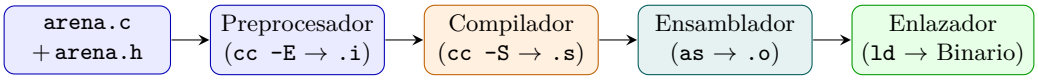


Figura 1. Tubería clásica de compilación separada en C: el preprocesador expande directivas sobre cada unidad de traducción (.i), el compilador emite código ensamblador (.s), el ensamblador genera objetos reubicables (.o) y el enlazador resuelve símbolos externos.

Consejo

Incluye siempre guardas de inclusión (`#ifndef MODULO_H` / `#define MODULO_H` / `#endif`) en todas tus cabeceras `.h` y nunca declares variables globales con definición en una cabecera sin `extern`.

Definición

Comportamiento Indefinido (Undefined Behavior, ISO C17 §3.4.3): uso de una construcción no portable o de datos erróneos para los cuales el estándar internacional no impone requisito alguno, permitiendo que el compilador optimice bajo la premisa axiomática de que dicha situación es inalcanzable.

Autoficha

¿Por qué comparar un índice `int i = -1` con `size_t n = 10` mediante `if (i < n)` evalúa a `false` en C debido a las conversiones aritméticas usuales?

1.1 Bloques A y B: Historia de C, modelo de máquina abstracta, GCC/Clang, CMake y flags de diagnóstico

Creado por Dennis Ritchie entre 1969 y 1973 en los Laboratorios Bell para reescribir el núcleo de UNIX (originalmente en ensamblador PDP-11), C fue estandarizado por ANSI en 1989 (C89/C90) y por ISO en las revisiones C99 (que introdujo `<stdint.h>`, `<stdbool.h>`, inicializadores designados y el calificador `restrict`), C11 (modelo de memoria multihilo y `_Atomic`), C17 y C23 (`nullptr`, `constexpr`, `typeof`, `<stdckdint.h>`). Como muestra la Figura \ref{fig:fig-c-translation-pipeline}, cada fichero `.c` junto con las cabeceras `.h` incluidas por `#include` constituye una *unidad de traducción* independiente. En ingeniería profesional, ningún programa en C debe compilarse sin activar las banderas estrictas de diagnóstico y los sanitizadores dinámicos: `clang -std=c17 -O2 -Wall -Wextra -Wpedantic -Werror -Wshadow -Wconversion -fsanitize=address,undefined`.

1.2 Bloque C: Tipos fundamentales, <stdint.h>, promociones aritméticas, operadores y control de flujo

Dado que el estándar C solo fija anchuras mínimas para `short`, `int`, `long` y `long long`, el código de sistemas portable utiliza siempre los tipos de anchura exacta de `<stdint.h>`: `uint8_t`, `uint16_t`, `uint32_t`, `uint64_t`, `int32_t`, `int64_t`, además de `size_t` (para tamaños y conteos de objetos en memoria) y `ptrdiff_t` (para diferencias entre punteros). Una regla crítica de la máquina abstracta de C es la asimetría entre enteros sin signo y con signo: mientras que los enteros sin signo (`uint32_t`, `size_t`) operan por definición en aritmética modular mód2^w, el desbordamiento de un entero **con signo** (`int32_t`) constituye **comportamiento indefinido** (*Undefined Behavior*, UB), lo que autoriza al optimizador a asumir que jamás ocurre y a eliminar comprobaciones ingenuas a posteriori como `if (a + b < a)`. Por ello, el desbordamiento con signo debe prevenirse **antes** de ejecutar la suma:

```
#include <stdint.h>
#include <stdbool.h>

bool safe_add_i32(int32_t a, int32_t b, int32_t *out) {
    if ((b > 0 && a > INT32_MAX - b) || (b < 0 && a < INT32_MIN - b)) {
        return false;
    }
    *out = a + b;
    return true;
}
```

En C23, la cabecera estándar `<stdckdint.h>` estandariza esta operación mediante la primitiva intrínseca `ckd_add(out, a, b)`.

Problema resuelto

Enunciado En una arquitectura de 32 bits para enteros `uint32_t` ($w = 32$, módulo $2^{32} = 4294967296$), un entero con signo con valor -100 se convierte implícitamente a `uint32_t` durante una comparación. Calcula el valor numérico exacto que toma tras la conversión según la regla $2^{32} - 100$.

Datos

- $w = 32\,\text{bits}$ — Anchura en bits de uint32_t
- $x = -100$ — Valor con signo original

Incógnita Magnitud objetivo solicitada en el enunciado

Modelo físico

Conversión a entero sin signo módulo 2^32 (ISO C §6.3.1.3)

$$U = 2^{32} - 100$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Sumamos 2^32 al valor negativo -100

$$U = 2^{32} - 100$$

$$\left((2)^{32} - 100\right)$$

Resultado

$Resultado = 4294967196. El valor resultante en uint32_t es 4294967196, razón por la cual ‘-W sign-conversion‘ es indispens$

Interpretación El valor resultante en uint32$_t$ es 4 294 967 196, razón por la cual `-Wsign-conversion` es indispensable.
Comprobación de los pasos

- Paso 1: 4294967196 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0003] (Básico, Numérico) Calcula el valor en `uint16_t` ($w = 16$) resultante de convertir -1 a sin signo ($2^{16} - 1$).
2 [PRB-0004] (Básico, Numérico) Un registro de control de 8 bits activa los bits 0, 3 y 7 ($\$2^0 + 2^3 + 2^7\$$). ¿Cuál es el valor decimal de la máscara `uint8_t`?
3 [PRB-0005] (Intermedio, Desarrollo por pasos) ¿Cuántos bytes ocupa en total una estructura con 4 campos `uint32_t` y 2 campos `uint64_t` perfectamente alineados ($4 \times 4 + 2 \times 8$)?
4 [PRB-0006] (Intermedio, Diseño) Si compilamos un proyecto en C formado por 18 unidades de traducción `.c` y solo modificamos un fichero `.c`, ¿cuántas unidades de traducción evita recompilar `make`/`ninja`?

Capítulo 2

Monografía de C (II – Bloques D y E): Punteros, aritmética de direcciones, memoria dinámica, structs y strict aliasing

*Este capítulo desarrolla los niveles intermedio y avanzado de C (Bloques D y E). Aquí desentrañamos la relación exacta entre arrays y punteros, la representación en memoria de estructuras y uniones, los riesgos de `malloc/free` individuales y por qué el patrón de ingeniería de sistemas por excelencia en C es el asignador por región (*Arena Allocator*).*

Consejo

Para reinterpretar los bits de un `float` como `uint32_t` o leer enteros de un paquete de red en C, utiliza siempre `memcpy(&destino, &origen, sizeof(destino))`: es 100 % seguro frente a strict aliasing y alineamiento, y el compilador lo optimiza a una sola instrucción de registro.

Definición

Regla de Alias Estricto (Strict Aliasing, ISO C17 §6.5): restricción semántica que prohíbe acceder a un valor almacenado mediante un puntero de un tipo distinto no relacionado (salvo tipos de carácter `unsigned char`), permitiendo al compilador mantener valores en registros sin recargarlos desde memoria tras escrituras a otros tipos.

Autoficha

¿Por qué tras ejecutar `free(ptr)` en C es una buena práctica defensiva asignar `ptr = NULL` si el puntero permanece aún en ámbito?

2.1 Bloque D: Punteros, arrays, cadenas terminadas en NUL, structs, unions y memoria dinámica

En C, un puntero `T *p` almacena la dirección virtual de un objeto de tipo `T`. La aritmética de punteros está escalada automáticamente por el tamaño del tipo apuntado: si `p` tiene dirección base A, la expresión `p + k` (y su equivalente por definición `&(p[k])`) evalúa a la dirección $A + k \cdot \text{sizeof}(T)$. El estándar ISO C permite formar y comparar un puntero únicamente dentro del mismo array asignado o exactamente una posición más allá del último elemento (`p + N`, denominado *one-past-the-end*), aunque desreferenciar `p + N` es comportamiento indefinido. Las cadenas en C son secuencias de bytes terminadas por el carácter nulo `'\0'` (`0x00`); calcular `strlen(s)` cuesta $O(n)$ porque debe recorrer los bytes hasta hallar el centinela, y usar funciones no acotadas permite desbordar el búfer si no se pasa explícitamente la capacidad máxima del destino.

2.2 Bloque E: Punteros a función, calificador restrict, strict aliasing y asignadores de región (Arena)

La regla de *alias estricto* (*strict aliasing*, ISO C17 §6.5 p7) estipula que un objeto en memoria solo puede ser accedido a través de un lvalue de tipo compatible con su tipo efectivo o a través de un puntero a carácter (`unsigned char *` o `char *`). Por tanto, convertir un `uint8_t *buf` arbitrario mediante `*(uint32_t *)buf` viola tanto el alias estricto como potencialmente el alineamiento de hardware; la forma conforme al estándar —que GCC y Clang compilan a una única instrucción máquina `mov` sin llamada real a función— consiste en usar `memcpy(&val, buf, sizeof(val))`. Asimismo, el calificador `restrict` en `void saxpy(size_t n, float a, const float *restrict x, float *restrict`

`y)` promete al compilador que las regiones apuntadas por `x` e `y` no se solapan, habilitando la vectorización SIMD automática:

```
#include <stddef.h>
#include <stdint.h>

void vector_scale_add(size_t n, float alpha, const float *restrict x, float *restrict y) {
    for (size_t i = 0; i < n; ++i) {
        y[i] += alpha * x[i];
    }
}
```

En sistemas de alto rendimiento (compiladores, servidores HTTP, videojuegos), en lugar de invocar miles de veces `malloc` y `free`, se reserva un bloque contiguo y se sirve memoria alineada incrementando un desplazamiento (`Arena Allocator`), liberando toda la región en $O(1)$ al finalizar la petición o fase.

Problema resuelto

Enunciado En el asignador por región `projects/c-arena-allocator`, el desplazamiento actual del búfer es `offset = 19` bytes y se solicita reservar una estructura de 40 B con requisito de alineamiento `align = 8` bytes. El asignador avanza primero `offset` al siguiente múltiplo de 8 (sumando 5 B de relleno hasta 24) y luego suma los 40 B del objeto. Calcula el nuevo valor exacto de `offset` tras la asignación.
Datos

- $offset_0 = 19\,\mathrm{B}$ — Desplazamiento actual antes de la reserva
- $pad = 5\,\mathrm{B}$ — Relleno para alcanzar el múltiplo de 8 (24 B)
- $size = 40\,\mathrm{B}$ — Tamaño del bloque solicitado

Incógnita Magnitud objetivo solicitada en el enunciado
Modelo físico
Avance de puntero con alineamiento en Arena Allocator

$$\mathrm{offset}_1 = (19 + 5) + 40$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Alineamos $19 + 5 = 24$ y sumamos los 40 bytes del bloque

$$\mathrm{offset}_1 = (19 + 5) + 40$$

$$((19 + 5) + 40)$$

Resultado

$$Resultado = 64. El desplazamiento final de la arena queda en 64 bytes.$$

Interpretación El desplazamiento final de la arena queda en 64 bytes.
Comprobación de los pasos

- Paso 1: 64 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [**PRB-0009**] **(Básico, Numérico)** Si un puntero `uint64_t *p` apunta a la dirección decimal 4096, ¿a qué dirección decimal apunta `p + 5` sabiendo que `sizeof(uint64_t) = 8`?
2 [**PRB-0010**] **(Básico, Numérico)** Una cadena en C contiene 31 caracteres visibles más su byte terminador `'\0'`. ¿Cuántos bytes mínimos debe reservar `malloc`?
3 [**PRB-0011**] **(Intermedio, Desarrollo por pasos)** En una arena con capacidad total de 4096 bytes, se reservan 12 nodos de 64 bytes ya alineados. ¿Cuántos bytes libres quedan en la arena?
4 [**PRB-0012**] **(Intermedio, Diseño)** Si un paquete binario de telemetría tiene 8 bytes de cabecera y transporta 16 muestras `int16_t` (2 bytes cada una), ¿cuántos bytes mide el paquete completo?

Capítulo 3

Monografía de C (III – Bloques F, G y H): Concurrencia C11/POSIX, Sanitizers, Ecosistema y Proyectos

Este capítulo cierra la monografía de C abordando el nivel experto (Bloque F), el ecosistema de construcción y auditoría (Bloque G) y los proyectos prácticos (Bloque H), incluyendo la verificación real del proyecto `projects/c-arena-allocator` *compilado con* `clang -std=c17 -fsanitize=address,undefined`.

Consejo

Nunca utilices `volatile` como sustituto de `<stdatomic.h>` o `pthread_mutex_t`: una carrera de datos sobre una variable `volatile` no atómica sigue siendo comportamiento indefinido según ISO C11/C17.

Definición

Memoria sombra en AddressSanitizer (ASan Shadow Memory): técnica de instrumentación de compilador donde cada bloque alineado de 8 bytes de la memoria de aplicación se proyecta mediante `(addr >> 3) + Offset` sobre 1 byte de memoria sombra que codifica si dichos 8 bytes son direccionables o pertenecen a una zona roja (*redzone*) envenenada.

Autoficha

¿Por qué el patrón `goto cleanup;` hacia adelante al final de una función en C mejora la seguridad de liberación de recursos frente a tener múltiples sentencias `return` dispersas que olvidan liberar descriptores previos?

3.1 Bloque F: Concurrencia C11 (<stdatomic.h>), pthreads, llamadas POSIX y programación defensiva

Un error histórico frecuente en C consiste en emplear el calificador `volatile` creyendo que proporciona sincronización entre hilos. En el estándar ISO C, `volatile` únicamente instruye al compilador para no omitir ni reordenar lecturas/escrituras respecto a registros de hardware mapeados en memoria (MMIO) o manejadores de señales (`sig_atomic_t`), pero **no** garantiza atomicidad ni emite barreras de coherencia de caché entre núcleos de CPU. Para concurrencia multihilo real, C11 introdujo `<stdatomic.h>` (`_Atomic uint64_t`, `atomic_fetch_add_explicit`, `memory_order_acquire`, `memory_order_release`). En sistemas UNIX/POSIX (IEEE Std 1003.1), la gestión de recursos que pueden fallar en cascada dentro de una misma función se estructura idiomáticamente mediante el patrón de limpieza centralizada con etiquetas `goto cleanup:`:

```
int process_file_buffer(size_t bytes) {
    int rc = -1;
    uint8_t *buf_a = malloc(bytes);
    if (!buf_a) goto fail_a;
    uint8_t *buf_b = malloc(bytes);
    if (!buf_b) goto fail_b;
    rc = 0; // operación exitosa
    free(buf_b);
fail_b:
    free(buf_a);
fail_a:
    return rc;
}
```

Este patrón asegura un único camino determinista de liberación en orden inverso de adquisición sin duplicar llamadas a `free`.

3.2 Bloques G y H: Ecosistema (CMake, Meson, pkg-config, Sanitizers) y los 4 proyectos en C

El ecosistema moderno de C se apoya en generadores de sistemas de construcción como `CMake` y `Meson` sobre el ejecutor `Ninja`, el sistema de metadatos de bibliotecas `pkg-config`, pruebas unitarias ligeras y, de forma obligatoria en integración continua, la tríada de sanitizadores de compilador: `AddressSanitizer` (detecta desbordamientos en pila/montículo/globales y *use-after-free* con una sobrecarga media inferior a 2×), `UndefinedBehaviorSanitizer` (detecta desbordamientos con signo, desplazamientos ilegales y punteros nulos o desalineados) y `ThreadSanitizer` (detecta carreras de datos). Los cuatro proyectos del bloque en C son: (1) una utilidad CLI compatible con POSIX (`wc/sha256`) usando E/S por bloques; (2) el asignador por arena y analizador de paquetes binarios con checksum `projects/c-arena-allocator`; (3) una tabla hash abierta genérica con sondeo lineal y factor de carga controlado; y (4) un servidor TCP multicliente no bloqueante basado en multiplexación de descriptores.

Problema resuelto

Enunciado Cuando un programa en C se compila con `-fsanitize=address` (AddressSanitizer), cada 8 B de memoria de aplicación requieren 1 B de memoria sombra (*shadow memory*), más zonas rojas (*redzones*) de 32 B alrededor de cada asignación. Si una aplicación reserva $N = 1000$ bloques de 64 B cada uno (con 32 B de zona roja por bloque), calcula cuántos bytes suman los bloques más sus zonas rojas más la memoria sombra correspondiente ($1/8$ del total).
Datos

- $N = 1000$ — Número de bloques asignados
- $S_{bloque} = 64\,\text{B}$ — Tamaño útil de cada bloque
- $S_{redzone} = 32\,\text{B}$ — Zona roja envenenada por bloque

Incógnita Magnitud objetivo solicitada en el enunciado
Modelo físico
Consumo de memoria bajo instrumentación AddressSanitizer (ASan)

$$M = 1000 \cdot (64 + 32) \cdot \frac{9}{8}$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Sumamos bloque + redzone (96000 B) y multiplicamos por 9/8 para incluir el byte sombra por cada 8 B

$$M = 1000 \cdot (64 + 32) \cdot \frac{9}{8}$$

$$\frac{((1000 \cdot (64 + 32)) \cdot 9)}{8}$$

Resultado

$$\mathit{Resultado} = 108000. \mathit{El conjunto instrumentado ocupa} 108000 \mathit{bytes} (96000 \mathit{B de bloques con redzones y} 12000 \mathit{B de memoria so}$$

Interpretación El conjunto instrumentado ocupa 108 000 bytes (96 000 B de bloques con redzones y 12 000 B de memoria sombra).
Comprobación de los pasos

- Paso 1: 108000 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0015] (Básico, Numérico) En AddressSanitizer, ¿cuántos bytes de memoria sombra corresponden a 64 000 bytes de memoria de aplicación ($64000 / 8$)?
2 [PRB-0016] (Básico, Numérico) En `projects/c-arena-allocator`, la cabecera de telemetría suma los campos `magic` (10), `version` (2), `flags` (3) y `payload_len` (25). ¿Cuál es el valor del checksum esperado?
3 [PRB-0017] (Intermedio, Desarrollo por pasos) Una función en C adquiere secuencialmente 5 recursos y falla al intentar adquirir el cuarto. ¿Cuántos recursos ya adquiridos debe liberar la cascada `goto cleanup` antes de retornar?
4 [PRB-0018] (Intermedio, Diseño) Un anillo de descriptores E/S tiene capacidad para 256 entradas de 16 bytes cada una. ¿Cuántos bytes mide el anillo completo mapeado en memoria?

Parte II

Monografía Autónoma de C++ (ISO/IEC 14882:2020 / C++23)

Capítulo 4

Monografía de C++ (I – Bloques A–D): De C++98 a C++20/23, RAII, Semántica de Movimiento y Punteros Inteligentes

Este capítulo inicia la monografía autónoma de C++ moderno (ISO C++20 / C++23). Lejos de tratar a C++ como «C con clases y `new`/`delete` manuales», enseñamos desde el primer día el paradigma moderno: propiedad automática determinista mediante RAII, vistas no propietarias (`std::string_view`, `std::span`), evaluación en tiempo de compilación (`constexpr`/`consteval`) y semántica de movimiento.

Consejo

En C++20/C++23, no escribas jamás `new` ni `delete` desnudos en código de aplicación: utiliza `std::make_unique<T>(...)` para propiedad única (cuyo tamaño y coste en ensamblador son idénticos a un puntero crudo `T*`) y pasa vistas `std::span<const T>` o `std::string_view` cuando una función no necesite adquirir propiedad.

Definición

Garantía fuerte frente a excepciones (Strong Exception Safety Guarantee): propiedad de una operación en C++ según la cual, si la operación termina lanzando una excepción, el estado observable del programa permanece idéntico al que tenía antes de iniciar la operación (semántica commit-or-rollback).

Autoficha

¿Por qué `std::vector<T>::push_back` se ve obligado a copiar en lugar de mover los elementos existentes al duplicar su capacidad si el constructor de movimiento `T(T&&)` no está declarado como `noexcept`?

4.1 Bloques A–C: Evolución C++11/14/17/20/23, toolchain, referencias, sobrecarga, constexpr y RAII

Diseñado por Bjarne Stroustrup desde 1979 y estandarizado por ISO en 1998, C++ experimentó una profunda modernización con C++11 (semántica de movimiento, lambdas, `auto`, `constexpr`, `std::unique_ptr`, modelo de memoria multihilo), C++14, C++17 (`std::optional`, `std::variant`, `std::string_view`, `if constexpr`) y C++20/C++23 (`concepts`, `std::ranges`, corrutinas, módulos, `std::span`, `std::expected`). El pilar arquitectónico de C++ es **RAII** (*Resource Acquisition Is Initialization*): todo recurso (memoria del montículo, socket, fichero, cerrojo `std::scoped_lock`) se vincula al ciclo de vida de un objeto alojado en la pila cuyo constructor establece la invariante y cuyo destructor `~T()` libera infaliblemente el recurso al salir del ámbito, incluso si se desenrolla la pila (*stack unwinding*) por el lanzamiento de una excepción.

4.2 Bloque D: Categorías de valores (lvalue/rvalue), Regla de Cero/Cinco, std::unique_ptr y std::span

C++11 clasificó toda expresión según su identidad y movibilidad en `lvalue` (tiene identidad persistente), `xvalue` (*expiring value*, tiene identidad pero sus recursos pueden ser robados) y `prvalue` (temporal puro). La **Regla de Cero** establece que las clases de lógica de negocio no deben escribir destructores, constructores de copia ni de movimiento manuales, sino componer miembros RAII (`std::vector`,

`std::string`, `std::unique_ptr`). Si una clase gestiona directamente un manejador crudo del sistema, aplica la **Regla de Cinco**: definir explícitamente destructor, constructor de copia, asignación por copia, constructor de movimiento (`noexcept`) y asignación por movimiento (`noexcept`):

```
#include <memory>
#include <span>
#include <numeric>

// std::span<const double> observa cualquier array, std::vector o std::array sin copiar:
constexpr double sum_window(std::span<const double> samples) noexcept {
    double acc = 0.0;
    for (double v : samples) { acc += v; }
    return acc;
}
```

Es fundamental marcar los constructores de movimiento como `noexcept`: `std::vector` solo mueve sus elementos durante una realojación si el constructor de movimiento promete `noexcept`; de lo contrario, cae de vuelta a copiar cada elemento para preservar la garantía fuerte frente a excepciones (*strong exception safety guarantee*).

Problema resuelto

Enunciado En una implementación estándar de C++ de 64 bits donde `sizeof(std::string) = 24` bytes y la optimización de cadena corta (*Small String Optimization*, SSO) almacena en línea cadenas de hasta 22 B sin alojar en el montículo, un programa crea un `std::vector<std::string>` con $N = 5000$ identificadores cortos de 12 B cada uno. Calcula cuántos bytes totales ocupan los 5000 objetos `std::string` dentro del búfer contiguo del vector (sabiendo que realizan 0 alojamientos adicionales fuera del vector).

Datos

- $N = 5000$ — Número de cadenas cortas de 12 B (<= 22 B SSO)
- $S_{str} = 24\,\mathrm{B}$ — Tamaño de cada objeto std::string en 64 bits

Incógnita Magnitud objetivo solicitada en el enunciado

Modelo físico

Huella de memoria con Small String Optimization (SSO) en C++

$$M = 5000 \cdot 24$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Multiplicamos las 5000 instancias por 24 B (sin alojamientos externos gracias a SSO)

$$M = 5000 \cdot 24$$

$$(5000 \cdot 24)$$

Resultado

$$\mathit{Resultado} = 120000. \mathit{Los5000identificadoresse almacenancontiguamenteen120000bytesconcerollamadasindividualesa$$

Interpretación Los 5000 identificadores se almacenan contiguamente en 120 000 bytes con cero llamadas individuales al montículo por cadena.

Comprobación de los pasos

- Paso 1: 120000 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0021] (Básico, Numérico) ¿Cuántos bytes ocupa `std::unique_ptr<double>` con deleción por defecto en una arquitectura de 64 bits?
2 [PRB-0022] (Básico, Numérico) Un `std::span<const int>` en C++20 con extensión dinámica almacena un puntero (8 B) y una longitud `size_t` (8 B). ¿Cuántos bytes ocupa en la pila?
3 [PRB-0023] (Intermedio, Desarrollo por pasos) Un `std::vector<int>` duplica su capacidad geométricamente desde 16 hasta 1024 elementos (`sizeof(int) = 4`). ¿Cuántos bytes reserva el búfer final de 1024 enteros?
4 [PRB-0024] (Intermedio, Diseño) Un bloque de control de `std::make_shared<T>` combina en una única reserva de montículo los dos contadores atómicos (16 B) más el objeto `T` de 48 B. ¿Cuántos bytes mide esa única reserva?

Capítulo 5

Monografía de C++ (II – Bloques E–H): Plantillas, Concepts C++20, std::ranges, Concurrencia, Corrutinas y Proyectos

Este capítulo completa la monografía de C++ desarrollando los Bloques E, F, G y H: desde la programación genérica con `concepts` y `std::ranges` hasta la concurrencia cooperativa con `std::jthread` y la verificación del planificador de tareas `projects/cpp-task-scheduler`.

Consejo

Sustituye `std::thread` por `std::jthread` en todo código C++20 nuevo: si una función lanza una excepción antes de llamar a `.join()` sobre un `std::thread` convencional, su destructor invoca `std::terminate()` abortando el proceso.

Definición

Concepto en C++20 (`concept`): predicado nombrado evaluado en tiempo de compilación que restringe el conjunto de argumentos de plantilla aceptados por una función o clase genérica y participa en la resolución de sobrecarga por subsunción lógica.

Autoficha

¿Cómo decide el compilador de C++20 qué sobrecarga de plantilla es más específica cuando dos funciones restringidas por `concepts` distintos son válidas para un mismo tipo (regla de subsunción)?

5.1 Bloques E y F: Plantillas, C++20 Concepts, std::ranges, std::jthread y corrutinas

Históricamente, las plantillas (*templates*) en C++ dependían de SFINAE (*Substitution Failure Is Not An Error*), produciendo diagnósticos de compilación crípticos cuando un tipo no cumplía una operación interna. C++20 resolvió este problema introduciendo **Concepts**: predicados en tiempo de compilación que especifican formalmente los requisitos sintácticos y semánticos de un parámetro de plantilla. En nuestro proyecto verificado `projects/cpp-task-scheduler`, el concepto `ExecutableTask` exige que cualquier tipo de tarea proporcione `id()`, `priority()` convertible a `int` y `execute()` convertible a `int64_t`:

```
#include <concepts>
#include <cstdint>

template <typename T>
concept ExecutableTask = requires(const T& t) {
    { t.id() } -> std::convertible_to<uint32_t>;
    { t.priority() } -> std::convertible_to<int>;
    { t.execute() } -> std::convertible_to<int64_t>;
};
```

Junto con los conceptos, la biblioteca `<ranges>` permite encadenar vistas perezosas mediante el operador tubería `|`, como `tasks | std::views::filter(...) | std::views::transform(...)`, y `<thread>` incorpora `std::jthread`, que se une automáticamente (`join()`) en su destructor e integra cancelación cooperativa mediante `std::stop_token`.

5.2 Bloques G y H: Ecosistema CMake/vcpkg/Conan/Clang-Tidy y los 4 proyectos (incluyendo cpp-task-scheduler)

En el Bloque G (Ecosistema), los proyectos profesionales de C++ se estructuran mediante `CMake` moderno basado en objetivos (`target_link_libraries`, `target_compile_features(tgt PUBLIC cxx_std_20)`), gestores de paquetes `vcpkg` o `Conan 2`, pruebas con `GoogleTest` o `Catch2`, benchmarking con `Google Benchmark` y auditoría estática continua con `clang-tidy` (perfiles `cppcoreguidelines-*`, `modernize-*`, `bugprone-*`). En el Bloque H, además de una biblioteca de álgebra lineal estática `constexpr`, un motor de eventos con `std::variant` y un servidor de red asíncrono, el lector cuenta con el proyecto completo `projects/cpp-task-scheduler`, que implementa una cola de prioridad determinista `TaskScheduler<Task>` verificada con `clang++ -std=c++20 -fsanitize=address,undefined`.

Problema resuelto

Enunciado En el proyecto `projects/cpp-task-scheduler`, se encolan cuatro tareas `(id, prioridad, base_cost)`: `(1, 2, 10)`, `(2, 9, 10)`, `(3, 5, 20)` y `(4, 9, 15)`. Si se ejecuta `execute_filtered(5)` (que filtra las tareas con `priority >= 5` y suma `base_cost * priority` de cada una), calcula el resultado exacto devuelto.
Datos

- $T_2 = prioridad 9, coste 10$ — Aporta 9 * 10 = 90
- $T_4 = prioridad 9, coste 15$ — Aporta 9 * 15 = 135
- $T_3 = prioridad 5, coste 20$ — Aporta 5 * 20 = 100

Incógnita Magnitud objetivo solicitada en el enunciado
Modelo físico
Filtrado y reducción en cpp-task-scheduler con std::views

$$R = 9 \cdot 10 + 9 \cdot 15 + 5 \cdot 20$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Descartamos la tarea 1 (prioridad $2 < 5$) y sumamos los productos de las tres restantes

$$R = 9 \cdot 10 + 9 \cdot 15 + 5 \cdot 20$$

$$(((9 \cdot 10) + (9 \cdot 15)) + (5 \cdot 20))$$

Resultado

$$Resultado = 325. El planificador devuelve exactamente 325, coincidiendo con el test ejecutable de `projects/cpp-task-sch$$

Interpretación El planificador devuelve exactamente 325, coincidiendo con el test ejecutable de `projects/cpp-task-scheduler`.
Comprobación de los pasos

- Paso 1: 325 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0027] (Básico, Numérico) En `cpp-task-scheduler`, ¿qué valor devuelve `ComputeTask{10, 7, 12}.execute()` (12×7)?
2 [PRB-0028] (Básico, Numérico) Una plantilla variádica `sum_all(10, 20, 30, 40)` utiliza una expresión pliegue (*fold expression* `(args + ...)`). ¿Cuál es el resultado evaluado en `constexpr`?
3 [PRB-0029] (Intermedio, Desarrollo por pasos) Un `std::variant<int, double, std::array<char, 32>>` almacena la alternativa mayor (32 B) más un índice alineado a 8 B. ¿Cuántos bytes ocupa la instancia?
4 [PRB-0030] (Intermedio, Diseño) Si una vista `std::views::iota(1, 1001) | std::views::filter(...) | std::views::take(5)` detiene la evaluación al encontrar los primeros 5 pares ($2, 4, 6, 8, 10$), ¿cuántos enteros del rango $[1, 1000]$ llegaron a inspeccionarse?

Parte III

Monografías Especializadas de Sistemas: Zig, Ensamblador, Ada y Fortran

Capítulo 6

Monografía de Zig (Bloques A–H): Asignadores explícitos, metaprogramación comptime, uniones de error y toolchain C

*Este capítulo presenta la monografía completa de **Zig** siguiendo los bloques $A-H$. Creado por Andrew Kelley, Zig se concibe como un sucesor moderno y transparente de C donde ninguna expresión oculta llamadas a funciones ni reservas de memoria, los tipos son valores de primera clase en tiempo de compilación (`comptime`) y el propio binario `zig` incluye un compilador C/C++ de compilación cruzada basado en Clang/LLVM.*

Consejo

En todas tus pruebas unitarias de Zig (`test "..." { ... }`), pasa siempre `std.testing.allocator` a las estructuras bajo prueba: el arnés de pruebas hará fallar automáticamente el test si queda un solo byte sin liberar o si ocurre doble liberación.

Definición

Limpieza condicional de error (`errdefer` en Zig): sentencia registrada en la pila léxica del bloque actual cuya expresión asociada se ejecuta si y solo si la función abandona dicho ámbito devolviendo un valor de error de su unión de retorno `!T`.

Autoficha

¿Qué ventaja arquitectónica aporta que `std.ArrayList` en Zig reciba un `Allocator` explícito en lugar de invocar un asignador global fijo como `std::vector` por defecto en C++ o `Vec<T>` en Rust estable?

6.1 Bloques A–D: Filosofía de Zig, toolchain integrada, tipos, rebanadas, uniones de error y defer/errdefer

Zig se rige por tres mandamientos de ingeniería de sistemas: (1) sin flujo de control oculto (no hay sobrecarga de operadores, ni constructores/destructores implícitos, ni excepciones); (2) sin asignaciones de memoria ocultas (toda función de la biblioteca estándar que necesita alojar en el montículo recibe explícitamente un parámetro `allocator: std.mem.Allocator`); y (3) sin preprocesador de macros textuales. Para gestionar recursos y errores con precisión quirúrgica, Zig combina tipos de unión de error (`AllocError![]u8`) con las palabras clave `try`, `catch`, `defer` (que se ejecuta al salir del bloque actual) y `errdefer` (que se ejecuta **únicamente** si el bloque actual retorna con un error):

```
const std = @import("std");

pub fn allocateInitializedRing(allocator: std.mem.Allocator, n: usize) ![]u64 {
    const buf = try allocator.alloc(u64, n);
    errdefer allocator.free(buf);
    for (buf, 0..) |*slot, idx| {
        slot.* = @as(u64, @intCast(idx)) * 10;
    }
    return buf;
}
```

Si cualquier paso posterior dentro de la función fallara y retornara un error mediante `try`, `errdefer` libera automáticamente `buf` sin necesidad de etiquetas `goto`.

6.2 Bloques E–H: Asignadores explícitos (Arena, FixedBuffer, GPA), comptime, FFI con C y proyectos

En lugar de plantillas o macros, el polimorfismo paramétrico en Zig consiste en funciones ordinarias que aceptan argumentos `comptime T: type` y devuelven un `type` (`fn BoundedQueue(comptime T: type, comptime cap: usize) type`). Gracias a que los asignadores se pasan como valores de interfaz `std.mem.Allocator`, el mismo contenedor `std.ArrayList(T)` puede ejecutarse sobre el asignador detector de fugas `std.testing.allocator` durante `zig test`, sobre un `std.heap.ArenaAllocator` por petición HTTP, o sobre un `std.heap.FixedBufferAllocator` respaldado por un array estático en la pila sin tocar jamás el sistema operativo.

Problema resuelto

Enunciado En un microservicio escrito en Zig, cada petición HTTP utiliza un `std.heap.FixedBufferAllocator` respaldado por un array en pila de $B = 16384$ B (16 KiB). Si el manejador aloja una cabecera parseada de 512 B y $N = 60$ cabeceras clave-valor de 128 B cada una, calcula cuántos bytes quedan disponibles en el `FixedBufferAllocator` antes de que devuelva `error.OutOfMemory`.

Datos

- $B = 16384\,\mathrm{B}$ — Capacidad total del búfer estático en pila
- $S_{hdr} = 512\,\mathrm{B}$ — Reserva inicial para el descriptor de petición
- $N_{kv}^{S} = 60,128B$ — 60 pares clave-valor de 128 B

Incógnita Magnitud objetivo solicitada en el enunciado

Modelo físico

Contabilidad lineal en FixedBufferAllocator de Zig

$$B_{\text{libre}} = 16384 - (512 + 60 \cdot 128)$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Restamos los 512 B y los 60 * 128 = 7680 B a la capacidad total de 16384 B

$$B_{\text{libre}} = 16384 - (512 + 60 \cdot 128)$$

$$(16384 - (512 + (60 \cdot 128)))$$

Resultado

$$Resultado = 8192.Quedanexactamente8192bytes(8KiB)libresenelbúferdelapetición.$$

Interpretación Quedan exactamente 8192 bytes (8 KiB) libres en el búfer de la petición.

Comprobación de los pasos

- Paso 1: 8192 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [**PRB-0033**] **(Básico, Numérico)** Una rebanada `[]const u32` en Zig de 64 bits consta de un puntero `[*]const u32` (8 B) y una longitud `usize` (8 B). ¿Cuántos bytes ocupa?
2 [**PRB-0034**] **(Básico, Numérico)** En Zig, el tipo entero de anchura arbitraria `u12` se aloja en memoria ocupando el siguiente número entero de bytes potencia de dos (`@sizeOf(u12)`). ¿Cuántos bytes ocupa en memoria?
3 [**PRB-0035**] **(Intermedio, Desarrollo por pasos)** Un `packed struct` de Zig contiene campos `u3`, `u5`, `u16` y `u8` empaquetados al bit exacto. ¿Cuántos bytes ocupa `(3 + 5 + 16 + 8) / 8`?
4 [**PRB-0036**] **(Intermedio, Diseño)** Si una función con 4 reservas secuenciales protegidas cada una por `errdefer` falla en la cuarta reserva, ¿cuántos bloques previos son liberados automáticamente por los `errdefer` ya registrados?

Capítulo 7

Monografía de Ensamblador ARM64 (AArch64) y x86-64 (Bloques A–H): Registros, ABI, Pila y SIMD

*Todo ingeniero de sistemas debe ser capaz de leer el código ensamblador emitido por el compilador (`clang -S -O2` u `objdump -d`), diagnosticar volcados de pila y escribir rutinas críticas vectorizadas o cambios de contexto de sistema operativo. Este capítulo ofrece el bloque completo A – H para las dos arquitecturas dominantes de 64 bits: **x86-64 (AMD64)** y **ARM64 (AArch64)**.*

System V AMD64 (x86-64)	**AAPCS64 (ARM64 / AArch64)**
Args 1–6: `rdi, rsi, rdx, rcx, r8, r9`	Args 1–8: `x0, x1, x2, x3, x4, x5, x6, x7`
Retorno: `rax` (`rdx`)	Retorno: `x0` (`x1`)
Callee-saved: `rbx, rbp, r12-r15`	Callee-saved: `x19-x28, x29(fp), x30(lr)`

Figura 2. Comparativa de paso de los primeros 6 argumentos enteros/punteros y valor de retorno entre la ABI System V x86-64 y la ABI AAPCS64 (ARM64).

Consejo

En lugar de escribir funciones enteras en ensamblador manual que dificultan las optimizaciones del compilador, prefiere funciones intrínsecas SIMD (`<arm_neon.h>` o `<immintrin.h>`) y verifica el ensamblador generado con `clang -O3 -S`.

Definición

Registros preservados por el llamado (Callee-saved registers): subconjunto de registros de la CPU (`rbx, rbp, r12–r15` en System V x86-64; `x19–x28, x29, x30` en AAPCS64) cuyo valor original debe ser restaurado por cualquier subrutina antes de ejecutar `ret` si dicha subrutina decide modificarlos.

Autoficha

¿Por qué en x86-64 al entrar en el cuerpo de una función tras ejecutar la instrucción `call` (que empuja 8 bytes de dirección de retorno en la pila), el registro `rsp` tiene un valor congruente con 8 módulo 16 y requiere restar $8 + 16k$ bytes antes de hacer otro `call`?

7.1 Bloques A–D: Arquitectura de registros x86-64 y ARM64 (AArch64), modos de direccionamiento y saltos

En **x86-64**, la CPU dispone de 16 registros de propósito general de 64 bits (`rax`, `rbx`, `rcx`, `rdx`, `rsi`, `rdi`, `rbp`, `rsp`, `r8–r15`) e instrucciones de longitud variable (de 1 a 15 bytes) que permiten combinar acceso a memoria y operación aritmética en una sola instrucción. En **ARM64 (AArch64)**, la arquitectura es estrictamente *load/store* con instrucciones de anchura fija de 32 bits (4 bytes) y 31 registros de propósito general de 64 bits (`x0–x30`, con sus mitades inferiores de 32 bits `w0–w30`), más el registro cero `xzr` y el puntero de pila `sp`. En ARM64, las operaciones aritméticas operan exclusivamente entre registros; la memoria solo se toca mediante instrucciones de carga y almacenamiento como `ldr`, `str`, `ldp` (cargar par de registros) y `stp` (almacenar par de registros).

7.2 Bloques E–H: Convenciones ABI (System V y AAPCS64), alineamiento de pila, SIMD (NEON/AVX) e inspección con objdump

Para que módulos compilados por distintos lenguajes (C, C++, Rust, Zig) puedan invocarse entre sí, deben respetar la **ABI** (*Application Binary Interface*) de la plataforma. Como resume la Figura \ref{fig:fig-abi-calling-conventions}, en la ABI System V x86-64 (Linux, macOS, BSD) los primeros 6 argumentos enteros o punteros se pasan en `rdi`, `rsi`, `rdx`, `rcx`, `r8`, `r9`, mientras que en AAPCS64 (ARM64) los primeros 8 se pasan en `x0`–`x7`. En ambas arquitecturas, el puntero de pila (`rsp` o `sp`) debe mantenerse alineado a un múltiplo de **16 bytes** en el punto de llamada a cualquier función (`call` o `bl`). Compare la misma función `uint64_t fma_u64(uint64_t a, uint64_t b, uint64_t c)` que calcula $a \cdot b + c$ en ambas arquitecturas:

```
; System V x86-64 (a=rdi, b=rsi, c=rdx -> retorno en rax):
fma_u64_x86:
    mov     rax, rdi
    imul    rax, rsi
    add     rax, rdx
    ret

; AAPCS64 ARM64 (a=x0, b=x1, c=x2 -> retorno en x0, una sola instrucción madd):
fma_u64_arm64:
    madd    x0, x0, x1, x2   ; x0 = (x0 * x1) + x2
    ret
```

En ARM64, el prólogo estándar de una función no hoja guarda el puntero de marco `x29` (`fp`) y el registro de enlace `x30` (`lr`) reservando un múltiplo de 16 bytes en una sola instrucción con pre-indexación: `stp x29, x30, [sp, #-16]!`.

Problema resuelto

Enunciado Una función en ensamblador ARM64 guarda en la pila el par `(x29, x30)` (16 B), dos registros preservados `(x19, x20)` (16 B) y necesita alojar un búfer local de 36 B. Como el hardware AArch64 exige que el puntero de pila `sp` permanezca siempre alineado a múltiplo de 16 B (redondeando los 36 B locales hacia arriba a 48 B), calcula cuántos bytes totales resta al registro `sp` el prólogo de la función.

Datos

- $S_{fplr} = 16\,\mathrm{B}$ — Par de registros (x29, x30)
- $S_{callee} = 16\,\mathrm{B}$ — Par de registros preservados (x19, x20)
- $S_{\updownarrow laligned} = 48\,\mathrm{B}$ — 36 B redondeados al siguiente múltiplo de 16 B

Incógnita Magnitud objetivo solicitada en el enunciado

Modelo físico

Cálculo del tamaño de marco de pila alineado a 16 bytes en AAPCS64

$$S_{\mathrm{frame}} = 16 + 16 + 48$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Sumamos los tres bloques alineados a 16 bytes

$$S_{\mathrm{frame}} = 16 + 16 + 48$$

$$((16 + 16) + 48)$$

Resultado

$$Resultado = 80. El prólogo reserva exactamente 80 bytes (múltiplo exacto de 16 : 5 * 16 = 80 B) en la pila.$$

Interpretación El prólogo reserva exactamente 80 bytes (múltiplo exacto de 16: 5 * 16 = 80 B) en la pila.

Comprobación de los pasos

- Paso 1: 80 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0039] (Básico, Numérico) En ARM64, cada instrucción máquina tiene una anchura fija de 32 bits (4 bytes). ¿Cuántos bytes ocupan en la sección `.text` una secuencia de 250 instrucciones?
2 [PRB-0040] (Básico, Numérico) Un registro vectorial NEON de 128 bits en ARM64 (`v0.4s`) procesa enteros o flotantes de 32 bits en paralelo. ¿Cuántos elementos de 32 bits opera por instrucción?
3 [PRB-0041] (Intermedio, Desarrollo por pasos) Una función en System V x86-64 recibe 9 argumentos enteros de 64 bits. Sabiendo que los primeros 6 van en registros y el resto se pasa por la pila (8 B cada uno), ¿cuántos bytes ocupan en la pila los argumentos restantes?
4 [PRB-0042] (Intermedio, Diseño) Un bucle procesa un array de 4096 flotantes `f32` utilizando instrucciones AVX-512 (16 flotantes `f32` por registro de 512 bits). ¿Cuántas iteraciones vectoriales ejecuta el bucle?

Capítulo 8

Monografías de Ada/SPARK (Sistemas Críticos) y Fortran Moderno (Cómputo Científico HPC) (Bloques A–H)

*Este capítulo cierra el Volumen IV con las monografías completas de dos lenguajes insustituibles en sus dominios de alta exigencia: **Ada 2022 / SPARK** en aviónica (DO-178C Nivel A), ferrocarriles y espacio, y **Fortran 2018/2023** en supercomputación (HPC), dinámica de fluidos computacional (CFD) y predicción meteorológica.*

Consejo

En Fortran moderno, comienza siempre todo módulo y programa con `implicit none` e importa `real64` (`dp`) e `int64` desde el módulo estándar `iso_fortran_env`; al anidar bucles sobre `A(i, j)`, sitúa siempre el índice `i` en el bucle más interno y `j` en el externo para recorrer la memoria secuencialmente.

Definición

Orden mayor por columnas (Column-Major Order en Fortran): disposición lineal en memoria de un array bidimensional $A(1..M, 1..N)$ según la cual el elemento $A(i, j)$ (con índices basados en 1) se ubica en el desplazamiento lineal $(j-1)\cdot M+(i-1)$, haciendo contiguos los elementos de una misma columna.

Autoficha

¿Por qué intercambiar el orden de los bucles `do i = 1, M; do j = 1, N` sobre `A(i, j)` en Fortran degrada severamente la tasa de aciertos de caché L1 respecto a `do j = 1, N; do i = 1, M`?

8.1 Monografía de Ada 2022 y SPARK (Bloques A–H): Tipado fuerte de dominio, contratos y sistemas críticos

Encargado por el Departamento de Defensa de EE. UU. (Jean Ichbiah, 1980; revisiones ISO Ada 95, 2005, 2012 y 2022), **Ada** fue diseñado para sistemas embebidos de misión crítica donde un defecto de software puede costar vidas humanas. En Ada, el ingeniero no declara variables como meros `int`, sino que define tipos nominalmente distintos con su rango físico exacto: `type Altitude_Meters is range -500 .. 50_000;` y `type Velocity_Knots is range 0 .. 2_500;`. Sumar una altitud con una velocidad es un error de compilación inmediato, y asignar un valor fuera de rango lanza `Constraint_Error`. En el subconjunto formalmente verificable **SPARK**, los aspectos `Pre`, `Post` y `Global` se traducen en obligaciones de prueba matemática descargadas por solucionadores SMT (`Z3`, `Alt-Ergo`) mediante `gnatprove`, demostrando la ausencia total de desbordamientos o divisiones por cero antes de compilar:

```
package Flight_Envelope with SPARK_Mode is
   type Altitude_M is range 0 .. 20_000;
   function Safe_Climb (Current : Altitude_M; Delta_H : Altitude_M) return Altitude_M
     with Pre  => Integer (Current) + Integer (Delta_H) <= 20_000,
          Post => Safe_Climb'Result = Current + Delta_H;
end Flight_Envelope;
```

Para concurrencia de tiempo real duro, el perfil **Ravenscar** restringe las tareas (`task`) y los objetos protegidos (`protected object`) al protocolo de techo de prioridad inmediata (*Immediate Ceiling Priority Protocol*), garantizando ausencia matemática de interbloqueos (*deadlocks*) y acotación estática del tiempo de respuesta.

8.2 Monografía de Fortran 2018/2023 (Bloques A–H): Arrays multidimensionales, vectorización y Coarrays

Por su parte, **Fortran** (John Backus, IBM, 1957; modernizado radicalmente en Fortran 90/95, 2003, 2008, 2018 y 2023) sigue siendo el estándar de oro en supercomputación científica por dos razones técnicas: primero, los arrays multidimensionales son ciudadanos de primera clase almacenados en **orden mayor por columnas** (*column-major*, donde `A(i, j)` es contiguo en memoria al variar el primer índice `i`); segundo, el estándar Fortran garantiza por defecto que los argumentos `intent(in)` e `intent(inout)` de una subrutina `pure` **no presentan alias entre sí** (a menos que tengan el atributo `pointer` o `target`), lo que permite a compiladores como `gfortran`, `ifx` o `nvfortran` vectorizar bucles matriciales con una agresividad superior a la de C sin `restrict`:

```
module numerics_mod
  use, intrinsic :: iso_fortran_env, only: dp => real64
  implicit none
contains
  pure subroutine laplacian_1d(u, d2u, inv_dx2)
    real(dp), intent(in)  :: u(:), inv_dx2
    real(dp), intent(out) :: d2u(:)
    integer :: n
    n = size(u)
    d2u(1) = 0.0_dp
    d2u(2:n-1) = (u(1:n-2) - 2.0_dp * u(2:n-1) + u(3:n)) * inv_dx2
    d2u(n) = 0.0_dp
  end subroutine laplacian_1d
end module numerics_mod
```

Además, Fortran 2008/2018 incorpora paralelismo de memoria distribuida directamente en la sintaxis del lenguaje mediante **Coarrays** (`real(dp) :: field(1000)[*]`), donde la notación entre corchetes `[imagen]` realiza transferencias RDMA de un solo lado entre nodos de un clúster sin llamadas verbosas a MPI.

Problema resuelto

Enunciado En un programa Fortran 2018, una matriz `real(real64) :: A(100, 200)` ($M = 100$ filas, $N = 200$ columnas, $S = 8$ B por elemento) está almacenada en orden mayor por columnas a partir del desplazamiento base 0. Calcula el desplazamiento exacto en bytes del elemento `A(25, 4)` usando la fórmula $8 \cdot ((4-1) \cdot 100 + (25-1))$.

Datos

- $M = 100$ — Número de filas por columna
- $i^j = 25, 4$ — Fila 25, columna 4 (base 1)
- $S = 8\,\text{B}$ — Tamaño de real(real64)

Incógnita Magnitud objetivo solicitada en el enunciado

Modelo físico

Direccionamiento Column-Major en Fortran

$$\text{Offset} = 8 \cdot ((4-1) \cdot 100 + (25-1))$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Calculamos el índice lineal (3 * 100 + 24 = 324) y multiplicamos por 8 bytes

$$\text{Offset} = 8 \cdot ((4-1) \cdot 100 + (25-1))$$

$$(8 \cdot (((4-1) \cdot 100) + (25-1)))$$

Resultado

$$Resultado = 2592. El elemento A(25, 4) se encuentra exactamente a 2592 bytes del inicio de la matriz.$$

Interpretación El elemento A(25, 4) se encuentra exactamente a 2592 bytes del inicio de la matriz.
Comprobación de los pasos

- Paso 1: 2592 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [**PRB-0045**] **(Básico, Numérico)** En Ada, el tipo `type Percent is range 0 .. 100;` comprende desde 0 hasta 100 ambos inclusive. ¿Cuántos valores distintos pertenecen al tipo (`Percent'Last - Percent'First + 1`)?
2 [**PRB-0046**] **(Básico, Numérico)** En Fortran, una rebanada vectorial `u(2:n-1)` para $n = 1002$ evalúa los puntos interiores de una malla 1D. ¿Cuántos puntos interiores actualiza la sentencia?
3 [**PRB-0047**] **(Intermedio, Desarrollo por pasos)** Un programa Coarray Fortran se ejecuta en 16 imágenes (`num_images() = 16`) y cada imagen aloja una malla local de 50000 flotantes `real64` (8 B). ¿Cuántos bytes suman las mallas de las 16 imágenes?
4 [**PRB-0048**] **(Intermedio, Diseño)** En un sistema de tiempo real Ada Ravenscar con periodo de marco mayor $T = 100$ ms, tres tareas periódicas consumen 15 ms, 22 ms y 18 ms por cada 100 ms. ¿Cuál es el porcentaje entero de utilización total de CPU?

Apéndice A

Notación

Símbolo	Significado	Unidades
alignof(T)	Requisito de alineamiento de dirección en C11/C23 y C++11/C++20	B
sizeof(T)	Tamaño en bytes de un objeto o tipo completo en C/C++	B
$k + p$	Aritmética de punteros escalada por k * sizeof(*p) bytes	dirección

Apéndice B

Hoja de fórmulas

Esta parte del borrador aún no tiene material.

Apéndice C

Soluciones

C.1 Monografía de C (I – Bloques A, B y C): Historia, modelo de traducción ISO C17/C23, toolchain y nivel principiante

Problema resuelto [PRB-0001]
En una arquitectura de 32 bits para enteros `uint32_t` ($w = 32$, módulo $2^{32} = 4294967296$), un entero con signo con valor -100 se convierte implícitamente a `uint32_t` durante una comparación. Calcula el valor numérico exacto que toma tras la conversión según la regla $2^{32} - 100$.
Respuesta

$$Resultado = 4294967196. El valor resultante en uint32_t es 4294967196, razón por la cual `-Wsign-conversion` es indispensable$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0003]
Calcula el valor en `uint16_t` ($w = 16$) resultante de convertir -1 a sin signo ($2^{16} - 1$).
Respuesta 65535
Ejercicio 2 [PRB-0004]
Un registro de control de 8 bits activa los bits 0, 3 y 7 ($\$2^0 + 2^3 + 2^7\$$). ¿Cuál es el valor decimal de la máscara `uint8_t`?
Respuesta 137
Ejercicio 3 [PRB-0005]
¿Cuántos bytes ocupa en total una estructura con 4 campos `uint32_t` y 2 campos `uint64_t` perfectamente alineados ($4 \times 4 + 2 \times 8$)?
Respuesta 32
Ejercicio 4 [PRB-0006]
Si compilamos un proyecto en C formado por 18 unidades de traducción `.c` y solo modificamos un fichero `.c`, ¿cuántas unidades de traducción evita recompilar `make`/`ninja`?
Respuesta 17

Cálculos reproducidos

$$\Big((2)^{32} - 100\Big) = 4294967196$$

C.2 Monografía de C (II – Bloques D y E): Punteros, aritmética de direcciones, memoria dinámica, structs y strict aliasing

Problema resuelto [PRB-0007]
En el asignador por región `projects/c-arena-allocator`, el desplazamiento actual del búfer es `offset = 19` bytes y se solicita reservar una estructura de 40 B con requisito de alineamiento `align = 8` bytes. El asignador avanza primero `offset` al siguiente múltiplo de 8 (sumando 5 B de relleno hasta 24) y luego suma los 40 B del objeto. Calcula el nuevo valor exacto de `offset` tras la asignación.
Respuesta

$$Resultado = 64. El desplazamiento final del la arena queda en 64 bytes.$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0009]

Si un puntero `uint64_t *p` apunta a la dirección decimal 4096, ¿a qué dirección decimal apunta `p + 5` sabiendo que `sizeof(uint64_t) = 8`?
Respuesta 4136
Ejercicio 2 [PRB-0010]
Una cadena en C contiene 31 caracteres visibles más su byte terminador `'\0'`. ¿Cuántos bytes mínimos debe reservar `malloc`?
Respuesta 32
Ejercicio 3 [PRB-0011]
En una arena con capacidad total de 4096 bytes, se reservan 12 nodos de 64 bytes ya alineados. ¿Cuántos bytes libres quedan en la arena?
Respuesta 3328
Ejercicio 4 [PRB-0012]
Si un paquete binario de telemetría tiene 8 bytes de cabecera y transporta 16 muestras `int16_t` (2 bytes cada una), ¿cuántos bytes mide el paquete completo?
Respuesta 40

Cálculos reproducidos

$((19 + 5) + 40) = 64$

C.3 Monografía de C (III – Bloques F, G y H): Concurrencia C11/POSIX, Sanitizers, Ecosistema y Proyectos

Problema resuelto [PRB-0013]
Cuando un programa en C se compila con `-fsanitize=address` (AddressSanitizer), cada 8 B de memoria de aplicación requieren 1 B de memoria sombra (*shadow memory*), más zonas rojas (*redzones*) de 32 B alrededor de cada asignación. Si una aplicación reserva $N = 1000$ bloques de 64 B cada uno (con 32 B de zona roja por bloque), calcula cuántos bytes suman los bloques más sus zonas rojas más la memoria sombra correspondiente ($1/8$ del total).
Respuesta

$$Resultado = 108000. El conjunto instrumentado ocupa 108000 bytes (96000 B de bloques con redzones y 12000 B de memoria som$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0015]
En AddressSanitizer, ¿cuántos bytes de memoria sombra corresponden a 64 000 bytes de memoria de aplicación ($64000 / 8$)?
Respuesta 8000
Ejercicio 2 [PRB-0016]
En `projects/c-arena-allocator`, la cabecera de telemetría suma los campos `magic` (10), `version` (2), `flags` (3) y `payload_len` (25). ¿Cuál es el valor del checksum esperado?
Respuesta 40
Ejercicio 3 [PRB-0017]
Una función en C adquiere secuencialmente 5 recursos y falla al intentar adquirir el cuarto. ¿Cuántos recursos ya adquiridos debe liberar la cascada `goto cleanup` antes de retornar?
Respuesta 3
Ejercicio 4 [PRB-0018]
Un anillo de descriptores E/S tiene capacidad para 256 entradas de 16 bytes cada una. ¿Cuántos bytes mide el anillo completo mapeado en memoria?
Respuesta 4096

Cálculos reproducidos

$\frac{((1000 \cdot (64+32)) \cdot 9)}{8} = 108000$

C.4 Monografía de C++ (I – Bloques A–D): De C++98 a C++20/23, RAII, Semántica de Movimiento y Punteros Inteligentes

Problema resuelto [PRB-0019]
En una implementación estándar de C++ de 64 bits donde `sizeof(std::string) = 24` bytes y la optimización de cadena corta (*Small String Optimization*, SSO) almacena en línea cadenas de hasta 22 B sin alojar en el montículo, un programa crea un `std::vector<std::string>` con $N = 5000$ identificadores cortos de 12 B cada uno. Calcula cuántos bytes totales ocupan los 5000 objetos `std::string` dentro del búfer contiguo del vector (sabiendo que realizan 0 alojamientos adicionales fuera del vector).
Respuesta

$$Resultado = 120000. Los 5000 identificadores se almacenan contiguamente en 120000 bytes con cero llamadas individuales al$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0021]
¿Cuántos bytes ocupa `std::unique_ptr<double>` con deleción por defecto en una arquitectura de 64 bits?
Respuesta 8
Ejercicio 2 [PRB-0022]
Un `std::span<const int>` en C++20 con extensión dinámica almacena un puntero (8 B) y una longitud `size_t` (8 B). ¿Cuántos bytes ocupa en la pila?
Respuesta 16
Ejercicio 3 [PRB-0023]
Un `std::vector<int>` duplica su capacidad geométricamente desde 16 hasta 1024 elementos (`sizeof(int) = 4`). ¿Cuántos bytes reserva el búfer final de 1024 enteros?
Respuesta 4096
Ejercicio 4 [PRB-0024]
Un bloque de control de `std::make_shared<T>` combina en una única reserva de montículo los dos contadores atómicos (16 B) más el objeto `T` de 48 B. ¿Cuántos bytes mide esa única reserva?
Respuesta 64

Cálculos reproducidos

$$(5000 \cdot 24) = 120000$$

C.5 Monografía de C++ (II – Bloques E–H): Plantillas, Concepts C++20, std::ranges, Concurrencia, Corrutinas y Proyectos

Problema resuelto [PRB-0025]
En el proyecto `projects/cpp-task-scheduler`, se encolan cuatro tareas `(id, prioridad, base_cost)`: `(1, 2, 10)`, `(2, 9, 10)`, `(3, 5, 20)` y `(4, 9, 15)`. Si se ejecuta `execute_filtered(5)` (que filtra las tareas con `priority >= 5` y suma `base_cost * priority` de cada una), calcula el resultado exacto devuelto.
Respuesta

$$Resultado = 325. El planificador devuelve exactamente 325, coincidiendo con el test ejecutable de `projects/cpp-task-sched$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0027]
En `cpp-task-scheduler`, ¿qué valor devuelve `ComputeTask{10, 7, 12}.execute()` (12×7)?

Respuesta 84

Ejercicio 2 [PRB-0028]
Una plantilla variádica `sum_all(10, 20, 30, 40)` utiliza una expresión pliegue (*fold expression* `(args + ...)`). ¿Cuál es el resultado evaluado en `constexpr`?

Respuesta 100

Ejercicio 3 [PRB-0029]
Un `std::variant<int, double, std::array<char, 32>>` almacena la alternativa mayor (32 B) más un índice alineado a 8 B. ¿Cuántos bytes ocupa la instancia?

Respuesta 40

Ejercicio 4 [PRB-0030]
Si una vista `std::views::iota(1, 1001) | std::views::filter(...) | std::views::take(5)` detiene la evaluación al encontrar los primeros 5 pares ($2, 4, 6, 8, 10$), ¿cuántos enteros del rango $[1, 1000]$ llegaron a inspeccionarse?

Respuesta 10

Cálculos reproducidos

$$(((9 \cdot 10) + (9 \cdot 15)) + (5 \cdot 20)) = 325$$

C.6 Monografía de Zig (Bloques A–H): Asignadores explícitos, metaprogramación comptime, uniones de error y toolchain C

Problema resuelto [PRB-0031]
En un microservicio escrito en Zig, cada petición HTTP utiliza un `std.heap.FixedBufferAllocator` respaldado por un array en pila de $B = 16384$ B (16 KiB). Si el manejador aloja una cabecera parseada de 512 B y $N = 60$ cabeceras clave-valor de 128 B cada una, calcula cuántos bytes quedan disponibles en el `FixedBufferAllocator` antes de que devuelva `error.OutOfMemory`.

Respuesta

$$Resultado = 8192. Quedan exactamente 8192 bytes (8 KiB) libres en el búfer de la petición.$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicio 1 [PRB-0033]
Una rebanada `[]const u32` en Zig de 64 bits consta de un puntero `[*]const u32` (8 B) y una longitud `usize` (8 B). ¿Cuántos bytes ocupa?

Respuesta 16

Ejercicio 2 [PRB-0034]
En Zig, el tipo entero de anchura arbitraria `u12` se aloja en memoria ocupando el siguiente número entero de bytes potencia de dos (`@sizeOf(u12)`). ¿Cuántos bytes ocupa en memoria?

Respuesta 2

Ejercicio 3 [PRB-0035]
Un `packed struct` de Zig contiene campos `u3`, `u5`, `u16` y `u8` empaquetados al bit exacto. ¿Cuántos bytes ocupa `(3 + 5 + 16 + 8) / 8`?

Respuesta 4

Ejercicio 4 [PRB-0036]
Si una función con 4 reservas secuenciales protegidas cada una por `errdefer` falla en la cuarta reserva, ¿cuántos bloques previos son liberados automáticamente por los `errdefer` ya registrados?

Respuesta 3

Cálculos reproducidos

$$(16384 - (512 + (60 \cdot 128))) = 8192$$

C.7 Monografía de Ensamblador ARM64 (AArch64) y x86-64 (Bloques A−H): Registros, ABI, Pila y SIMD

Problema resuelto [PRB-0037]
Una función en ensamblador ARM64 guarda en la pila el par `(x29, x30)` (16 B), dos registros preservados `(x19, x20)` (16 B) y necesita alojar un búfer local de 36 B. Como el hardware AArch64 exige que el puntero de pila `sp` permanezca siempre alineado a múltiplo de 16 B (redondeando los 36 B locales hacia arriba a 48 B), calcula cuántos bytes totales resta al registro `sp` el prólogo de la función.
Respuesta

$$Resultado = 80. El prólogo reserva exactamente 80 bytes (múltiplo exacto de 16 : 5 * 16 = 80 B) en la pila.$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0039]
En ARM64, cada instrucción máquina tiene una anchura fija de 32 bits (4 bytes). ¿Cuántos bytes ocupan en la sección `.text` una secuencia de 250 instrucciones?
Respuesta 1000
Ejercicio 2 [PRB-0040]
Un registro vectorial NEON de 128 bits en ARM64 (`v0.4s`) procesa enteros o flotantes de 32 bits en paralelo. ¿Cuántos elementos de 32 bits opera por instrucción?
Respuesta 4
Ejercicio 3 [PRB-0041]
Una función en System V x86-64 recibe 9 argumentos enteros de 64 bits. Sabiendo que los primeros 6 van en registros y el resto se pasa por la pila (8 B cada uno), ¿cuántos bytes ocupan en la pila los argumentos restantes?
Respuesta 24
Ejercicio 4 [PRB-0042]
Un bucle procesa un array de 4096 flotantes `f32` utilizando instrucciones AVX-512 (16 flotantes `f32` por registro de 512 bits). ¿Cuántas iteraciones vectoriales ejecuta el bucle?
Respuesta 256

Cálculos reproducidos

$((16 + 16) + 48) = 80$

C.8 Monografías de Ada/SPARK (Sistemas Críticos) y Fortran Moderno (Cómputo Científico HPC) (Bloques A−H)

Problema resuelto [PRB-0043]
En un programa Fortran 2018, una matriz `real(real64) :: A(100, 200)` ($M = 100$ filas, $N = 200$ columnas, $S = 8$ B por elemento) está almacenada en orden mayor por columnas a partir del desplazamiento base 0. Calcula el desplazamiento exacto en bytes del elemento `A(25, 4)` usando la fórmula $8 \cdot ((4 - 1) \cdot 100 + (25 - 1))$.
Respuesta

$$Resultado = 2592. El elemento A(25, 4) se encuentra exactamente a 2592 bytes del inicio de la matriz.$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0045]
En Ada, el tipo `type Percent is range 0 .. 100;` comprende desde 0 hasta 100 ambos inclusive. ¿Cuántos valores distintos pertenecen al tipo (`Percent'Last - Percent'First + 1`)?
Respuesta 101
Ejercicio 2 [PRB-0046]

En Fortran, una rebanada vectorial `u(2:n-1)` para $n = 1002$ evalúa los puntos interiores de una malla 1D. ¿Cuántos puntos interiores actualiza la sentencia?
Respuesta 1000
Ejercicio 3 [PRB-0047]
Un programa Coarray Fortran se ejecuta en 16 imágenes (`num_images() = 16`) y cada imagen aloja una malla local de 50000 flotantes `real64` (8 B). ¿Cuántos bytes suman las mallas de las 16 imágenes?
Respuesta 6400000
Ejercicio 4 [PRB-0048]
En un sistema de tiempo real Ada Ravenscar con periodo de marco mayor $T = 100$ ms, tres tareas periódicas consumen 15 ms, 22 ms y 18 ms por cada 100 ms. ¿Cuál es el porcentaje entero de utilización total de CPU?
Respuesta 55

Cálculos reproducidos

$(8 \cdot (((4 - 1) \cdot 100) + (25 - 1))) = 2592$

Apéndice D

Auditoría de fuentes

Estado de las fuentes: PENDING. 32 pendientes. Los conflictos almacenados quedan fuera (0). Las fuentes que la guía almacenada no cita quedan fuera. Recuento de no citadas: 0.

¶1: dos testimonios.
¶2: dos testimonios.
¶3: dos testimonios.
¶4: dos testimonios.
¶5: dos testimonios.
¶6: dos testimonios.
¶7: dos testimonios.
¶8: dos testimonios.
¶9: dos testimonios.
¶10: dos testimonios.
¶11: dos testimonios.
¶12: dos testimonios.
¶13: dos testimonios.
¶14: dos testimonios.
¶15: dos testimonios.
¶16: dos testimonios.
¶17: dos testimonios.
¶18: dos testimonios.
¶19: dos testimonios.
¶20: dos testimonios.
¶21: dos testimonios.
¶22: dos testimonios.
¶23: dos testimonios.
¶24: dos testimonios.
¶25: dos testimonios.
¶26: dos testimonios.
¶27: dos testimonios.
¶28: dos testimonios.
¶29: dos testimonios.
¶30: dos testimonios.
¶31: dos testimonios.
¶32: dos testimonios.
¶33: dos testimonios.
¶34: dos testimonios.
¶35: dos testimonios.
¶36: dos testimonios.
¶37: dos testimonios.
¶38: dos testimonios.
¶39: dos testimonios.
¶40: dos testimonios.
¶41: dos testimonios.
¶42: dos testimonios.
¶43: dos testimonios.
¶44: dos testimonios.
¶45: dos testimonios.
¶46: dos testimonios.
¶47: dos testimonios.
¶48: dos testimonios.

Referencia Normativa src-iso-c17 (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶1, ¶2, ¶3, ¶4, ¶5, ¶6, ¶7, ¶8, ¶9, ¶10, ¶11, ¶12, ¶13, ¶14, ¶15, ¶16, ¶17, ¶18, ¶31, ¶32, ¶33, ¶34, ¶35, ¶36.
Referencia Normativa src-bryant-ohallaron (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶1, ¶2, ¶3, ¶4, ¶5, ¶6, ¶7, ¶8, ¶9, ¶10, ¶11, ¶12, ¶19, ¶20, ¶21, ¶22, ¶23, ¶24, ¶31, ¶32, ¶33, ¶34, ¶35, ¶36, ¶37, ¶38, ¶39, ¶40, ¶41, ¶42.
Referencia Normativa src-posix-2017 (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶13, ¶14, ¶15, ¶16, ¶17, ¶18.
Referencia Normativa src-iso-cpp20 (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶19, ¶20, ¶21, ¶22, ¶23, ¶24, ¶25, ¶26, ¶27, ¶28, ¶29, ¶30.
Referencia Normativa src-cormen-clrs (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶25, ¶26, ¶27, ¶28, ¶29, ¶30.
Referencia Normativa src-patterson-hennessy (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶37, ¶38, ¶39, ¶40, ¶41, ¶42.
IEEE Standard for Floating-Point Arithmetic (IEEE 754-2019) / Wikipedia Summary (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶43, ¶44, ¶45, ¶46, ¶47, ¶48.
Referencia Normativa src-knuth-taocp (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶43, ¶44, ¶45, ¶46, ¶47, ¶48.
REV-0001: cap01-c-fundamentos-modelo-compilacion-tipos-principiante scientific
REV-0002: cap02-c-punteros-memoria-dinamica-estructuras-aliasing scientific
REV-0003: cap03-c-experto-atomicos-posix-sanitizers-ecosistema-proyectos scientific
REV-0004: cap04-cpp-fundamentos-raii-move-semantics-smart-pointers scientific
REV-0005: cap05-cpp-templates-concepts-ranges-concurrencia-proyectos scientific
REV-0006: cap06-monografia-zig-asignadores-comptime-interoperabilidad-c scientific
REV-0007: cap07-monografia-ensamblador-arm64-x86-64-abi-simd scientific
REV-0008: cap08-monografias-ada-spark-tiempo-real-y-fortran-cientifico scientific

Apéndice E

Plan de estudio

Monografía Autónoma de C (ISO/IEC 9899:2018 / C23)

✓Monografía de C (I – Bloques A, B y C): Historia, modelo de traducción ISO C17/C23, toolchain y nivel principiante

- Comprender la evolución histórica de C desde UNIX en Bell Labs (Dennis Ritchie, 1972) hasta los estándares ANSI C89, C99, C11, C17 y C23.
- Dominar las 8 fases de traducción del estándar ISO C, el preprocesador, la compilación separada por unidades de traducción y el enlazado ELF/Mach-O.
- Utilizar con precisión los tipos enteros de anchura exacta (<stdint.h>), promociones enteras, operadores bit a bit y estructuras de control.

2 conceptos planificados.

✓Monografía de C (II – Bloques D y E): Punteros, aritmética de direcciones, memoria dinámica, structs y strict aliasing

- Dominar la semántica de punteros, el decaimiento de arrays a punteros, punteros a funciones y la aritmética escalada de direcciones.
- Gestionar el ciclo de vida en el montículo con malloc, calloc, realloc y free, y construir asignadores lineales de región (Arena Allocators).
- Aplicar la regla de alias estricto (strict aliasing), el calificador restrict y la lectura binaria segura mediante memcpy.

2 conceptos planificados.

✓Monografía de C (III – Bloques F, G y H): Concurrencia C11/POSIX, Sanitizers, Ecosistema y Proyectos

- Programar concurrencia segura en C mediante <stdatomic.h>, pthreads, descriptores de fichero POSIX y mmap.
- Integrar AddressSanitizer (ASan), UndefinedBehaviorSanitizer (UBSan), ThreadSanitizer (TSan) y analizadores estáticos en CMake/Meson.
- Auditar y extender el proyecto completo ‘projects/c-arena-allocator‘ junto con los 4 proyectos integradores en C.

2 conceptos planificados.

Monografía Autónoma de C++ (ISO/IEC 14882:2020 / C++23)

✓Monografía de C++ (I – Bloques A–D): De C++98 a C++20/23, RAII, Semántica de Movimiento y Punteros Inteligentes

- Comprender la filosofía de C++ moderno (Bjarne Stroustrup): tipos definidos por el usuario con coste cero y determinismo RAII.
- Aplicar la Regla de Cero, la Regla de Cinco, categorías de valores (lvalue, prvalue, xvalue) y semántica de movimiento (std::move).

- Gestionar recursos dinámicos sin fugas mediante std::unique_ptr, std::shared_ptr, std::weak_ptr, std::span y std::string_view.

2 conceptos planificados.

✓Monografía de C++ (II – Bloques E–H): Plantillas, Concepts C++20, std::ranges, Concurrencia, Corrutinas y Proyectos

- Escribir código genérico restringido semánticamente mediante C++20 Concepts (‘requires‘), plantillas variádicas y ‘if constexpr‘.
- Componer tuberías perezosas de vistas con ‘std::ranges‘ y ‘std::views‘ evitando copias intermedias.
- Dominar la concurrencia en C++20 (‘std::jthread‘, ‘std::stop_token‘, ‘std::atomic‘, ‘std::latch‘), el ecosistema CMake/Conan/vcpkg y el proyecto ‘projects/cpp-task-scheduler‘.

2 conceptos planificados.

Monografías Especializadas de Sistemas: Zig, Ensamblador, Ada y Fortran

✓Monografía de Zig (Bloques A–H): Asignadores explícitos, metaprogramación comptime, uniones de error y toolchain C

- Comprender los principios de diseño de Zig: ausencia de flujo de control oculto, ausencia de asignaciones implícitas de montículo y ausencia de preprocesador.
- Programar con uniones de error (‘!T‘), limpieza diferida determinista (‘defer‘ y ‘errdefer‘), rebanadas con longitud y asignadores explícitos (‘std.mem.Allocator‘).
- Utilizar ‘comptime‘ para tipos genéricos y evaluación en tiempo de compilación, e integrar cabeceras C con ‘@cImport‘.

2 conceptos planificados.

✓Monografía de Ensamblador ARM64 (AArch64) y x86-64 (Bloques A–H): Registros, ABI, Pila y SIMD

- Distinguir la arquitectura RISC de carga/almacenamiento AArch64 (ARM64) frente a la arquitectura CISC de longitud variable x86-64.
- Dominar las convenciones de llamada ABI System V AMD64 (‘rdi, rsi, rdx, rcx, r8, r9‘ y retorno en ‘rax‘) y AAPCS64 (‘x0–x7‘ y retorno en ‘x0‘).
- Analizar el prólogo y epílogo de marcos de pila, el alineamiento obligatorio de 16 bytes del puntero de pila (‘sp‘/‘rsp‘) e instrucciones vectoriales SIMD.

2 conceptos planificados.

✓Monografías de Ada/SPARK (Sistemas Críticos) y Fortran Moderno (Cómputo Científico HPC) (Bloques A–H)

- Diseñar tipos escalares con rangos restringidos, contratos ejecutables (‘Pre‘, ‘Post‘, ‘Type_Invariant‘) y tareas críticas bajo el perfil Ravenscar en Ada 2022 y SPARK.
- Comprender cómo el demostrador formal GNATprove en SPARK demuestra estáticamente la ausencia de errores en tiempo de ejecución (AoRTE).
- Programar cálculo numérico de alto rendimiento en Fortran 2018/2023 mediante operaciones matriciales intrínsecas, ‘intent(in/out)‘, módulos y paralelismo Coarray.

2 conceptos planificados.

Bibliografía

[1] Alan M. Turing – On Computable Numbers, with an Application to the Entscheidungsproblem (1936). https://www.cs.virginia.edu/~robins/Turing_Paper_1936.pdf

[2] John von Neumann – First Draft of a Report on the EDVAC (1945). https://web.mit.edu/STS.035/www/PDFs/edvac.pdf

[3] IEEE Standard for Floating-Point Arithmetic (IEEE 754-2019) / Wikipedia Summary. https://en.wikipedia.org/wiki/IEEE_754

[4] RFC 3629 – UTF-8, a transformation format of ISO 10646 (IETF Standard). https://datatracker.ietf.org/doc/html/rfc3629

[5] ACM / IEEE-CS / AAAI Computer Science Curricula 2023 (CS2023). https://csed.acm.org/

[6] MIT OpenCourseWare 6.006 – Introduction to Algorithms. https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/

[7] The Rust Programming Language (Klabnik & Nichols – Official Rust Documentation). https://doc.rust-lang.org/book/

[8] The Rust Reference – Memory Model, Type System, Lifetimes, and Unsafe Rust. https://doc.rust-lang.org/reference/

[9] The Rustonomicon – The Dark Arts of Unsafe Rust. https://doc.rust-lang.org/nomicon/

[10] C Programming Language Reference (ISO/IEC 9899 C17/C23 – cppreference). https://en.cppreference.com/w/c

[11] C++ Reference (ISO/IEC 14882 C++20/C++23 – cppreference). https://en.cppreference.com/w/cpp

[12] The Python Language Reference & Standard Library Documentation (Python 3.12–3.14). https://docs.python.org/3/reference/index.html

[13] ECMA-262 – ECMAScript 2024 Language Specification. https://tc39.es/ecma262/

[14] The TypeScript Handbook – Structural Typing, Narrowing, and Type Manipulation. https://www.typescriptlang.org/docs/handbook/intro.html

[15] Microsoft .NET & C# Language Specification (ECMA-334 / .NET 8 & 9). https://learn.microsoft.com/en-us/dotnet/csharp/

[16] The Java Language Specification & JVM Specification (Java SE 21 LTS). https://docs.oracle.com/javase/specs/jls/se21/html/index.html

[17] The Go Programming Language Specification & Memory Model. https://go.dev/ref/spec

[18] Kotlin Language Documentation & Coroutines Guide (Kotlin 2.0). https://kotlinlang.org/docs/home.html

[19] The Swift Programming Language (Swift 6.0 Official Book). https://docs.swift.org/swift-book/documentation/the-swift-programming-language/

[20] E. F. Codd – A Relational Model of Data for Large Shared Data Banks (1970) & PostgreSQL Docs. https://www.postgresql.org/docs/current/

[21] SQLite 3.54 Transactional SQL Database Engine Documentation. https://www.sqlite.org/docs.html

[22] GNU Bash Reference Manual & IEEE Std 1003.1 POSIX Shell Specification. https://www.gnu.org/software/bash/manual/bash.html

[23] WHATWG HTML Living Standard & W3C CSS Specifications. https://html.spec.whatwg.org/multipage/

[24] RFC 8259 – The JavaScript Object Notation (JSON) Data Interchange Format & YAML 1.2.2. https://datatracker.ietf.org/doc/html/rfc8259

[25] IEEE Computer Society – Guide to the Software Engineering Body of Knowledge (SWEBOK v4.0). https://www.computer.org/education/bodies-of-knowledge/software-engineering

[26] Referencia Normativa src-iso-c17. https://doc.rust-lang.org/reference/src-iso-c17.html

[27] Referencia Normativa src-bryant-ohallaron. https://doc.rust-lang.org/reference/src-bryant-ohallaron.html

[28] Referencia Normativa src-posix-2017. https://doc.rust-lang.org/reference/src-posix-2017.html

[29] Referencia Normativa src-iso-cpp20. https://doc.rust-lang.org/reference/src-iso-cpp20.html

[30] Referencia Normativa src-cormen-clrs. https://doc.rust-lang.org/reference/src-cormen-clrs.html

[31] Referencia Normativa src-patterson-hennessy. https://doc.rust-lang.org/reference/src-patterson-hennessy.html

[32] Referencia Normativa src-knuth-taocp. https://doc.rust-lang.org/reference/src-knuth-taocp.html